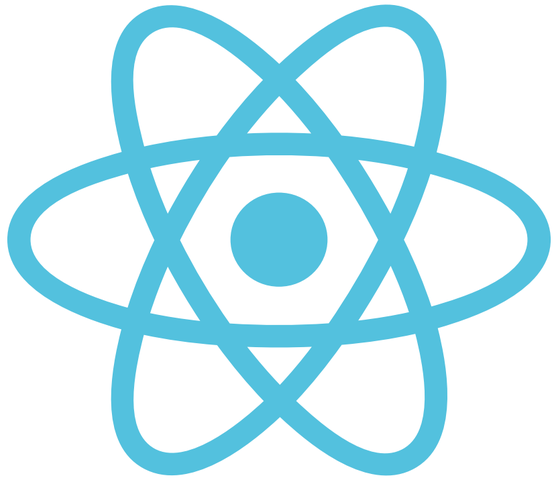
By: Mahmoud Abdelaziz

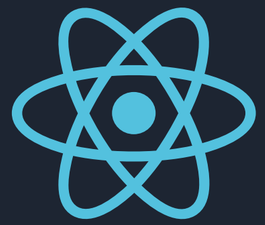
REACT JS



CONTENT

- JSX
- jsx syntax limitations
- Class vs Fn. Component
- State & Props & Children Props
- Life Cycle Methods
- Events
- Form
- Routing

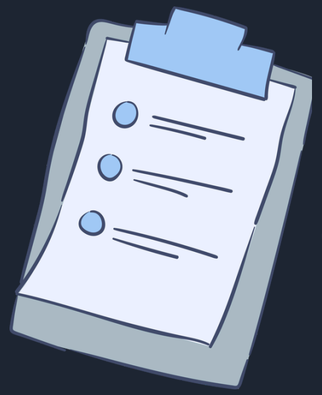
- Styling
- Pure Components
- React hooks



- Context API
- React.Lazy
- Lifting State up
- uuid , conditional Render
- Call Api

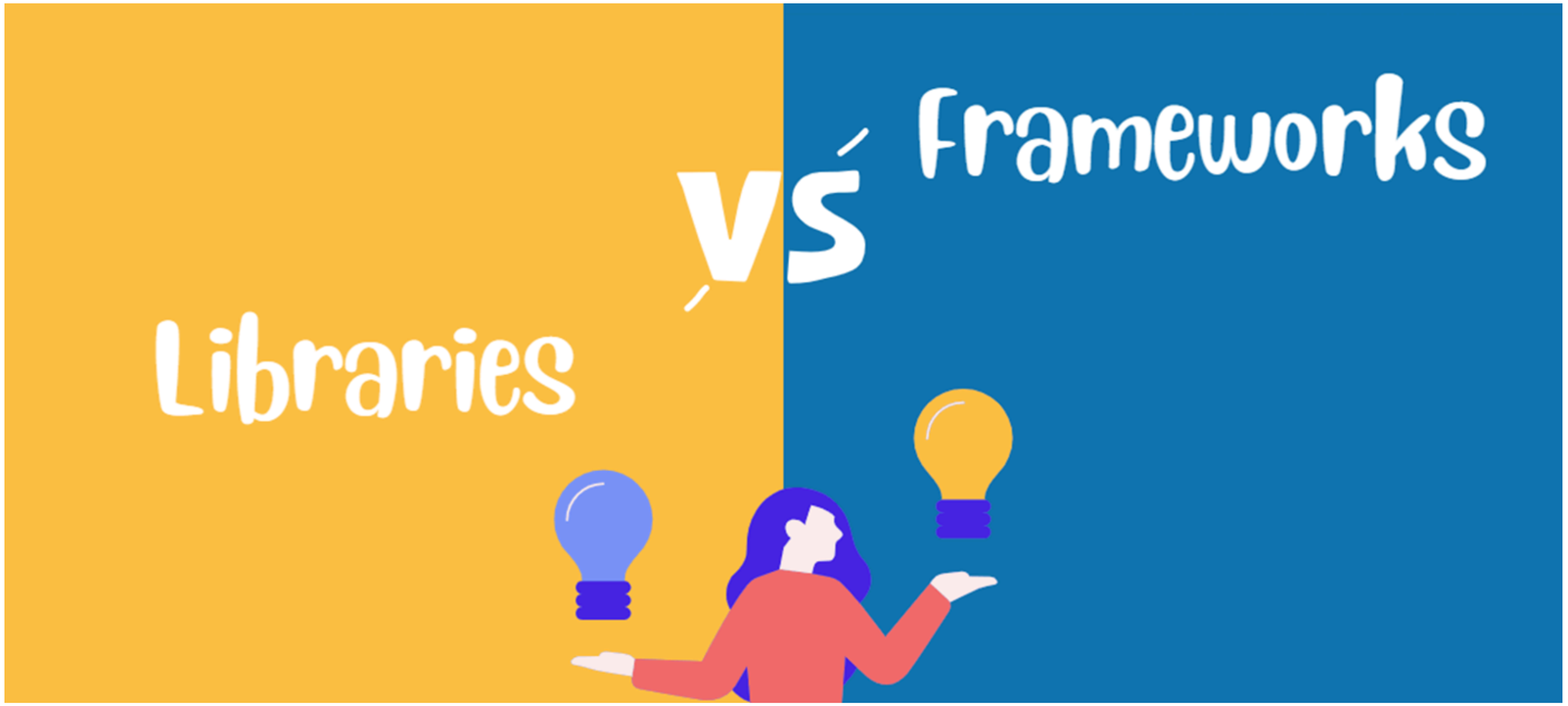
DAY 1

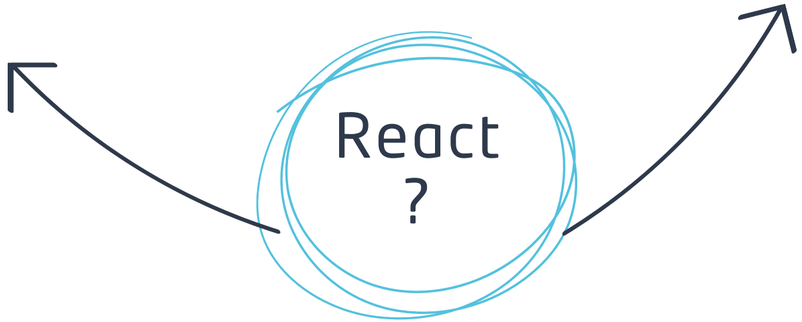
- Introduction
  - Framework Vs Library
  - Tools Behind
  - Why React
  - MPA Vs SPA
- How React Engine Works
- Create React Project
- Components
  - Class Comp
  - Function Comp
  - State And Props



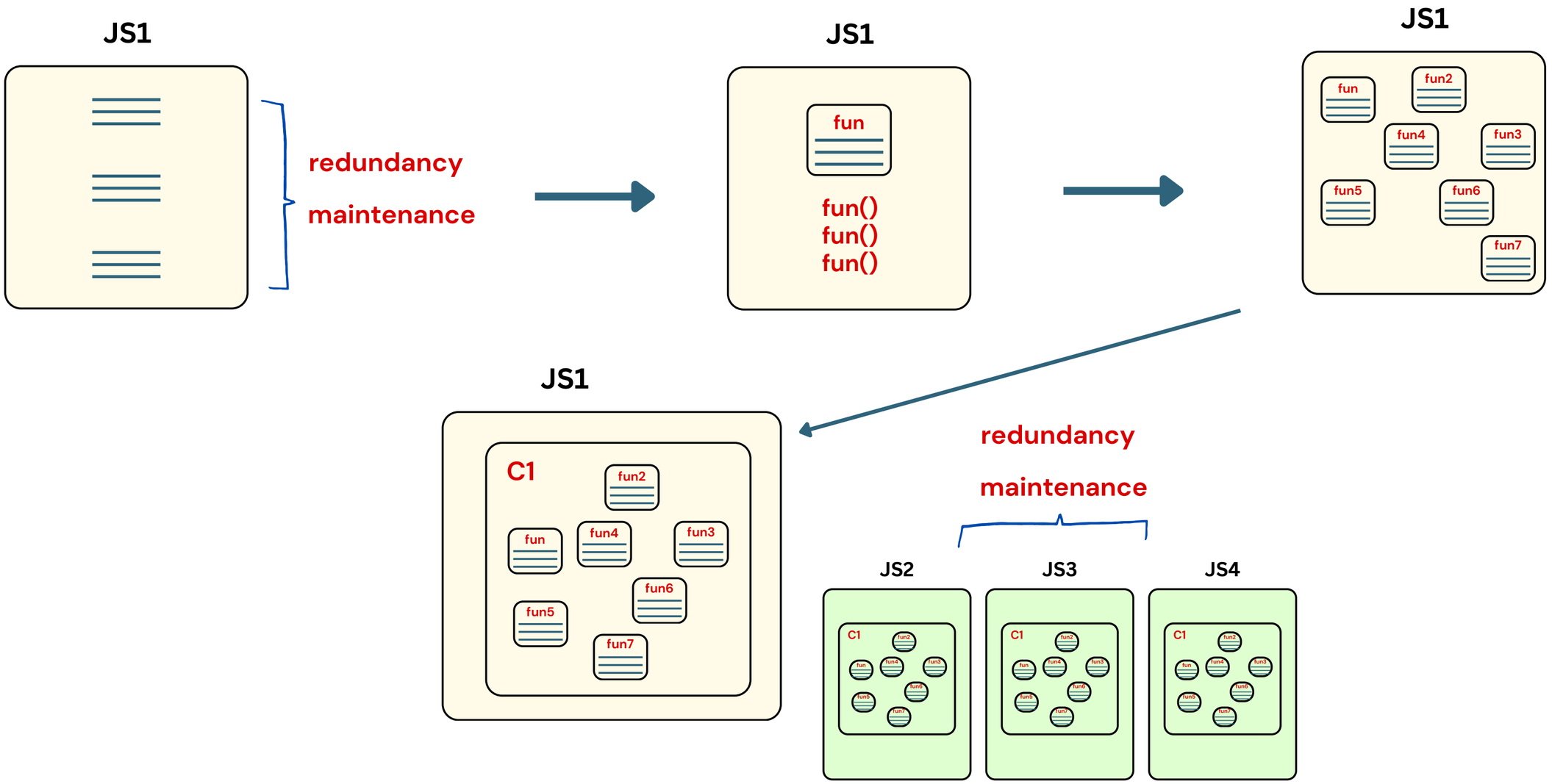
Agenda

Framework vs Library

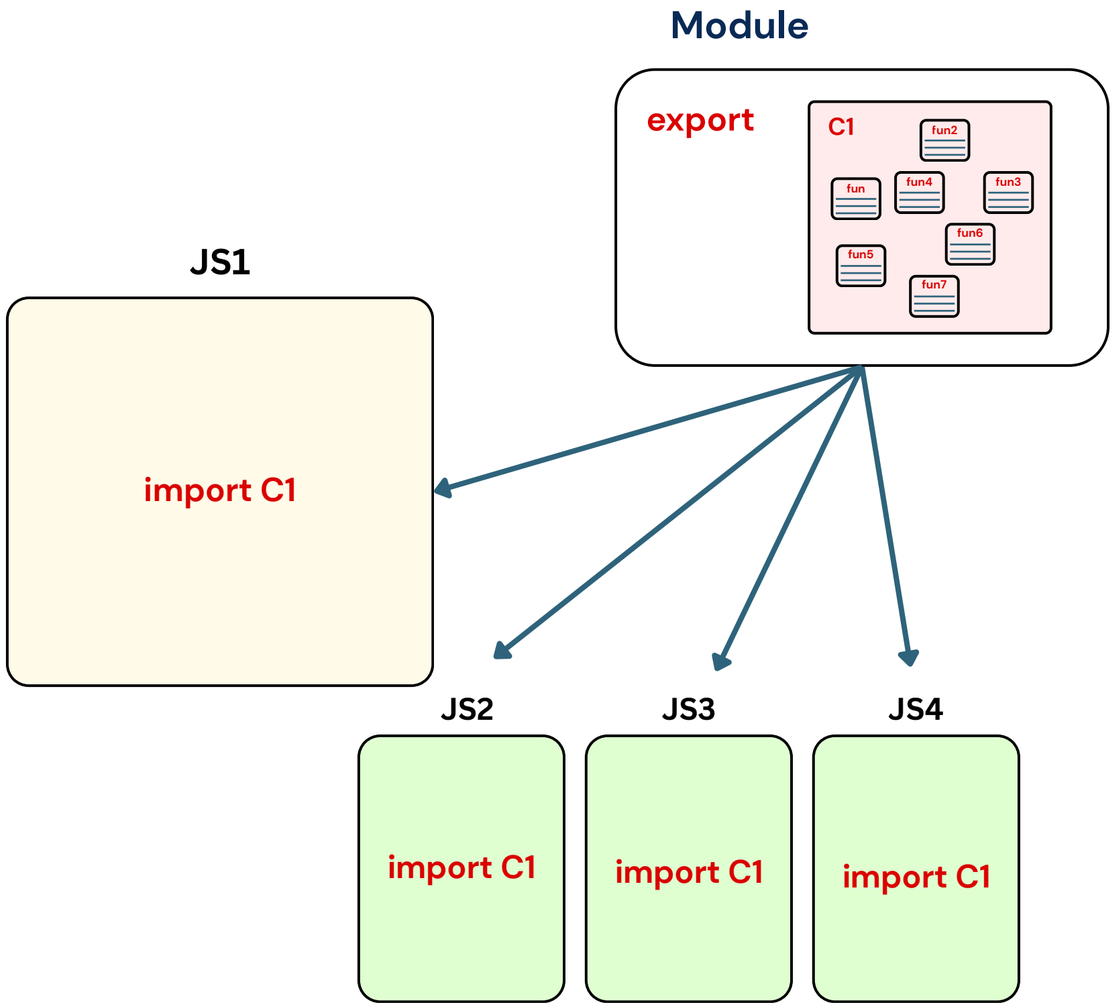




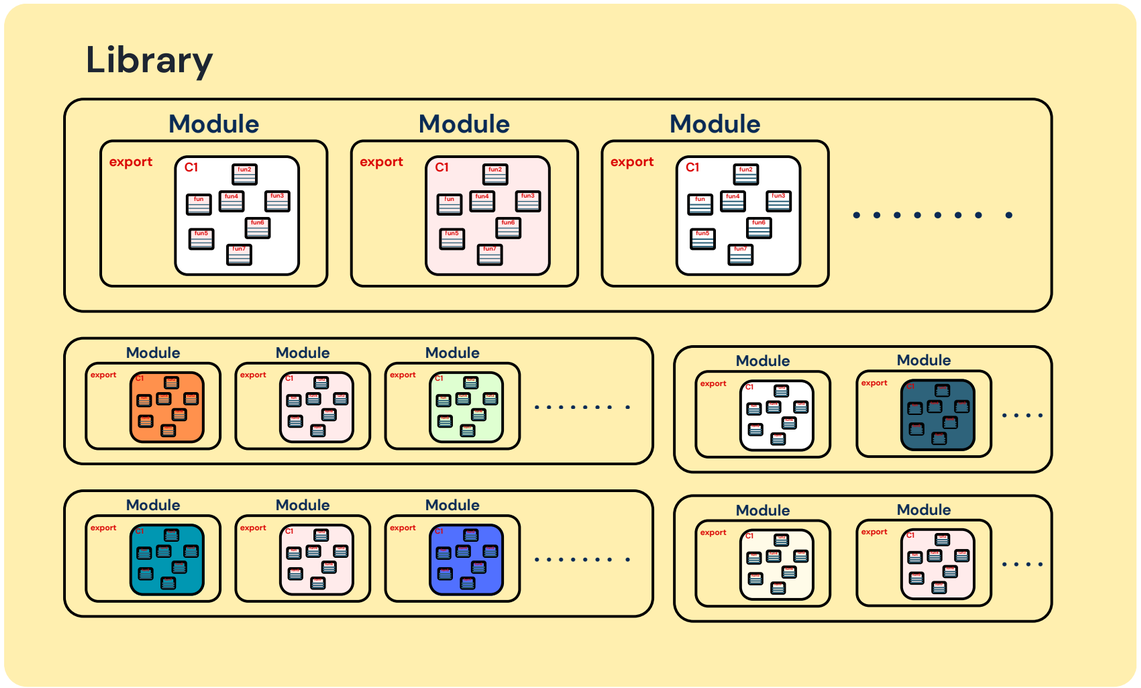
Framework vs Library

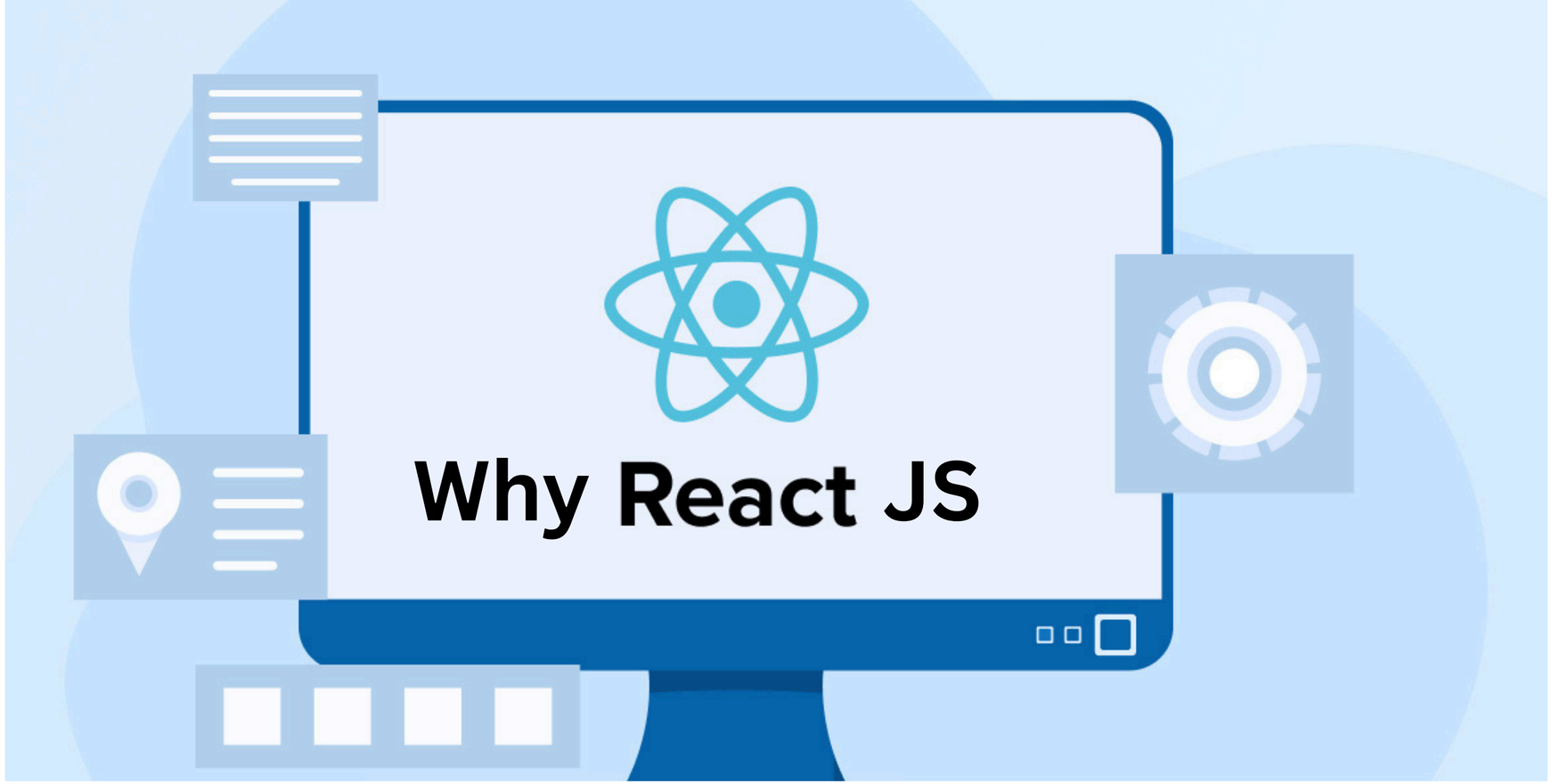


Framework vs Library



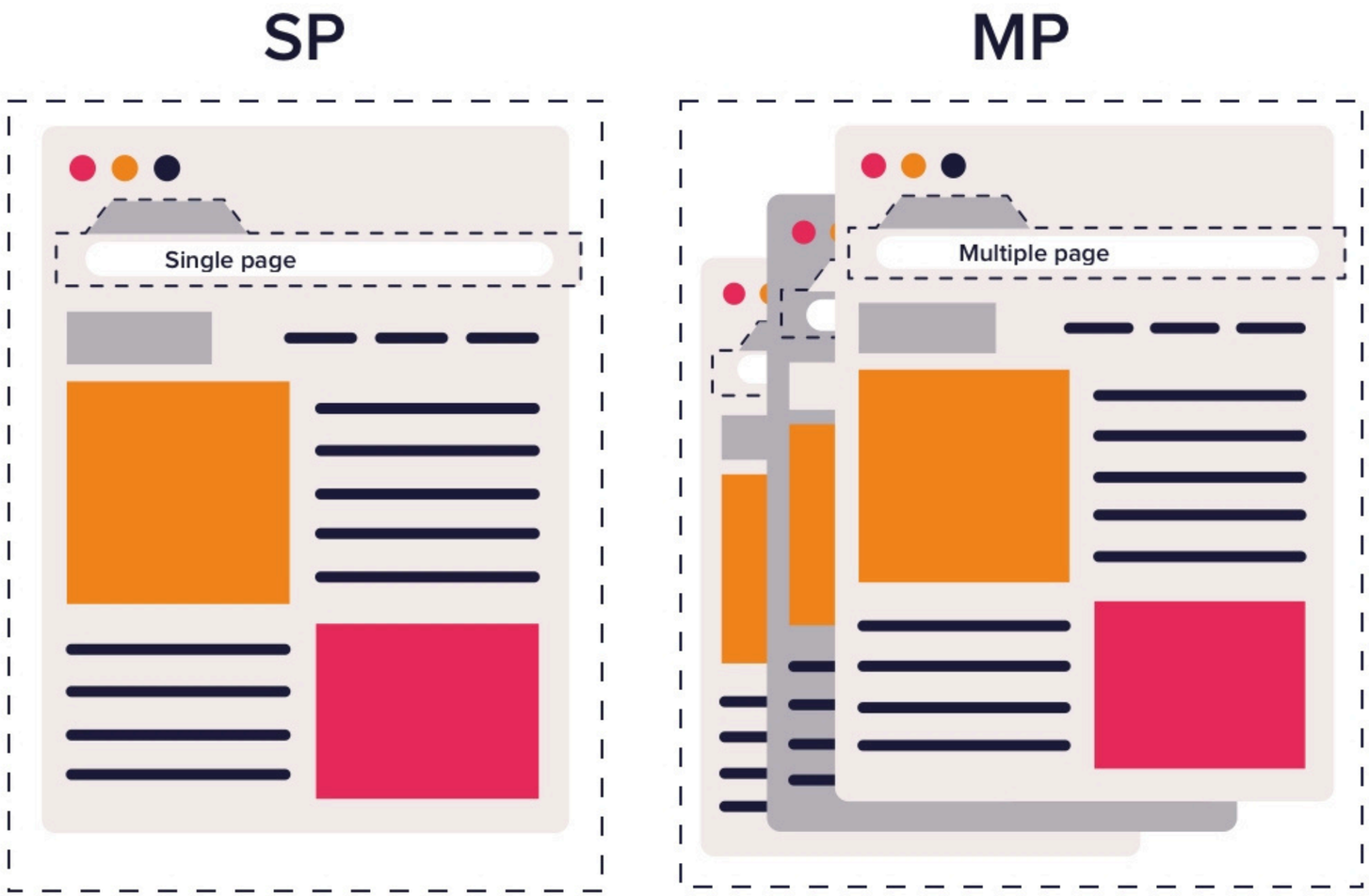
Framework



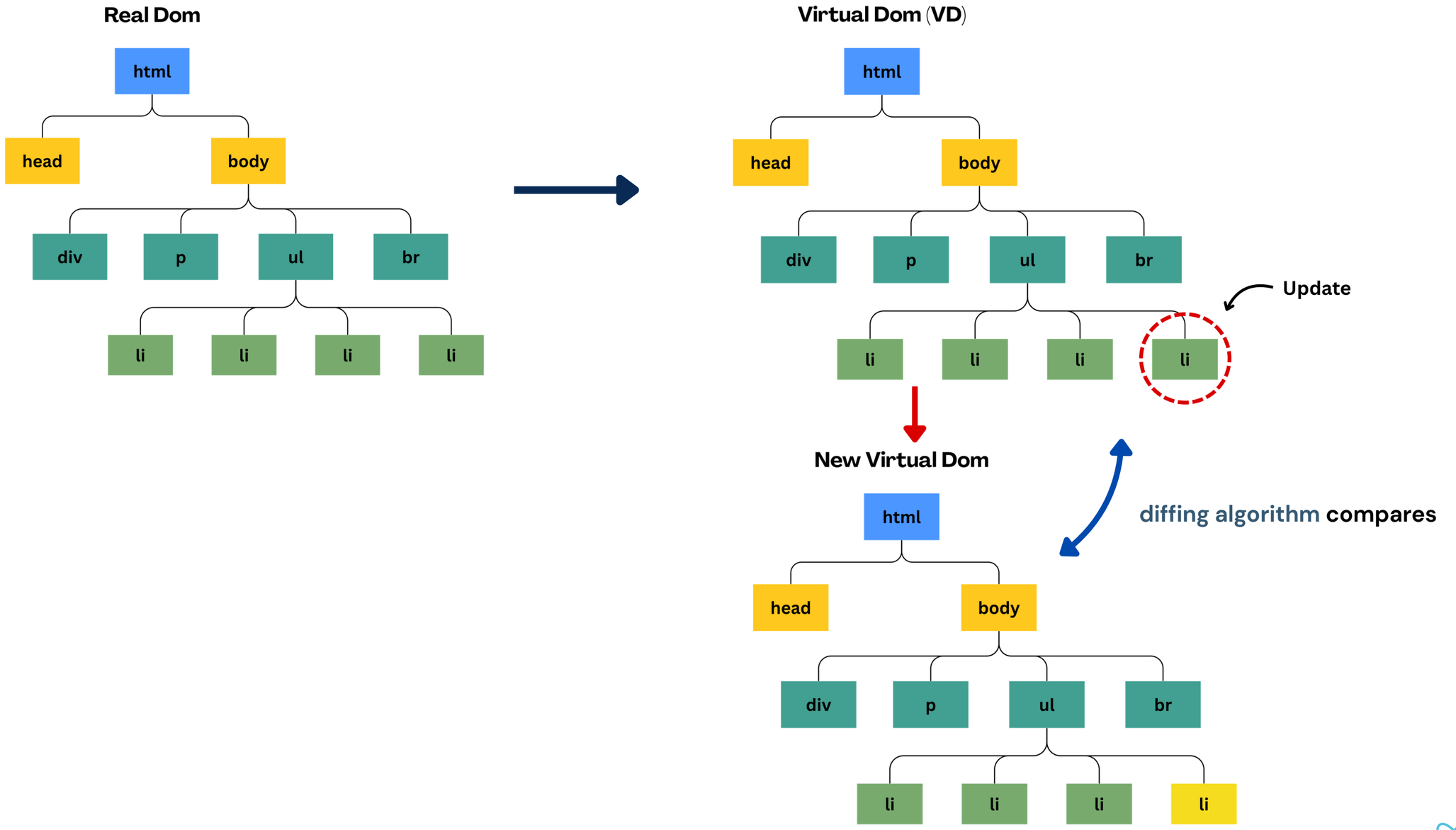


React.js was created by **Jordan Walke**, a software engineer at Facebook.
He developed React in **2011**.
React was first used in Facebook's news feed and **2012** on Instagram
before being open-sourced in **2013**. Since then, it has grown to become one of the most popular libraries for building web applications.

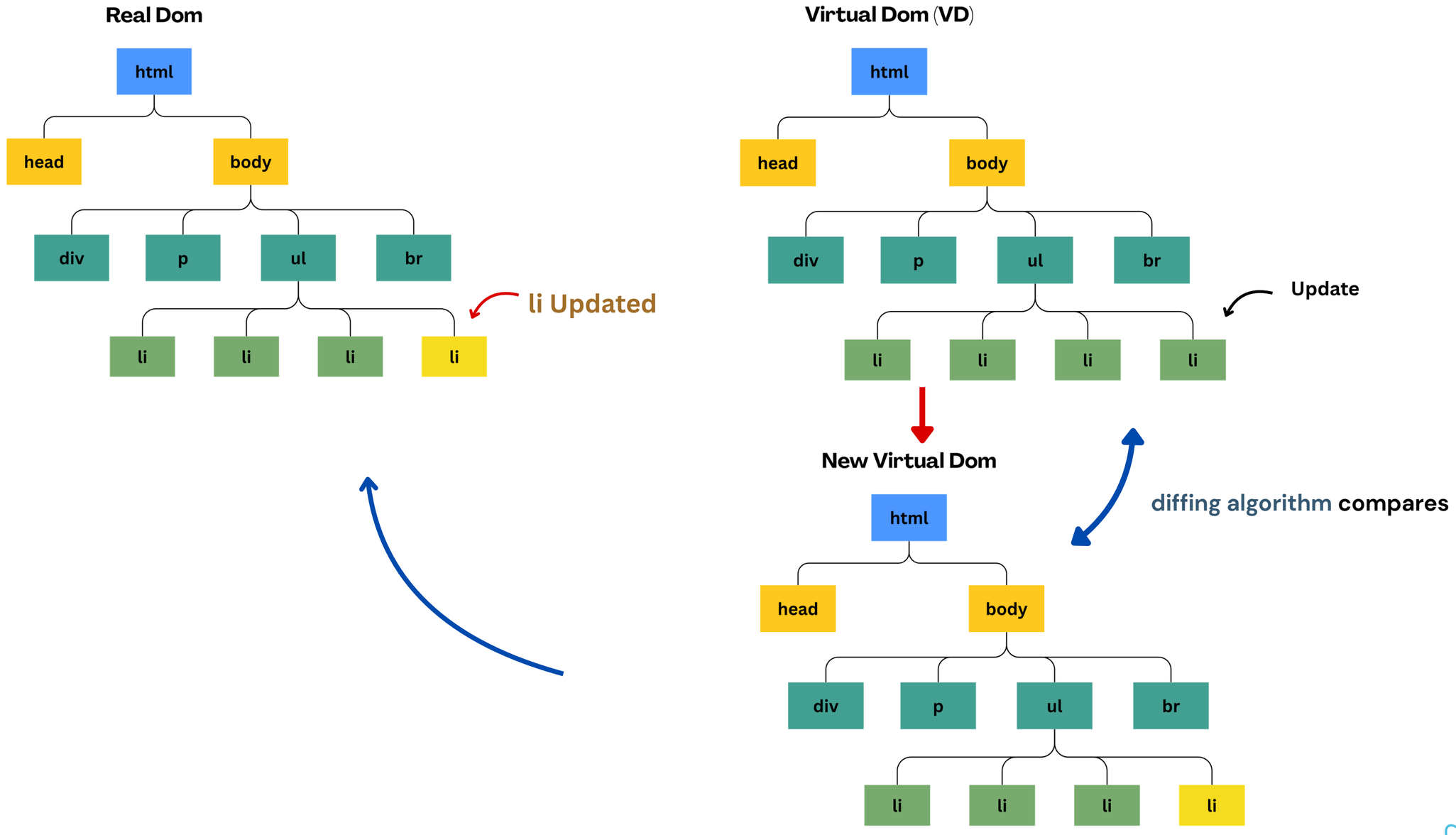
MPA Vs SPA



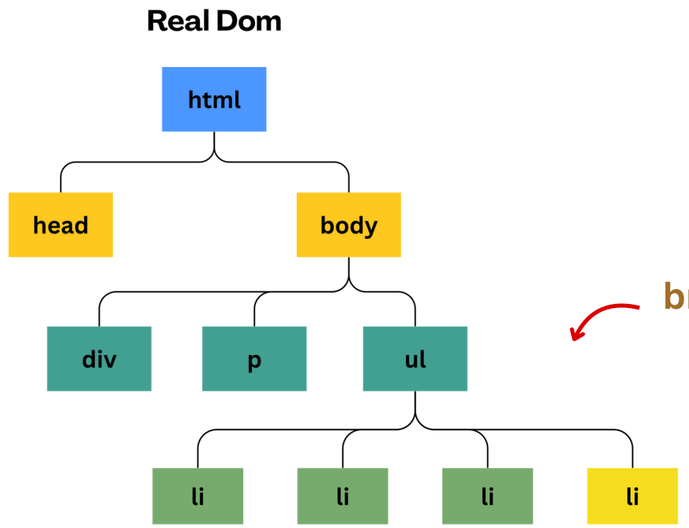
How React Works



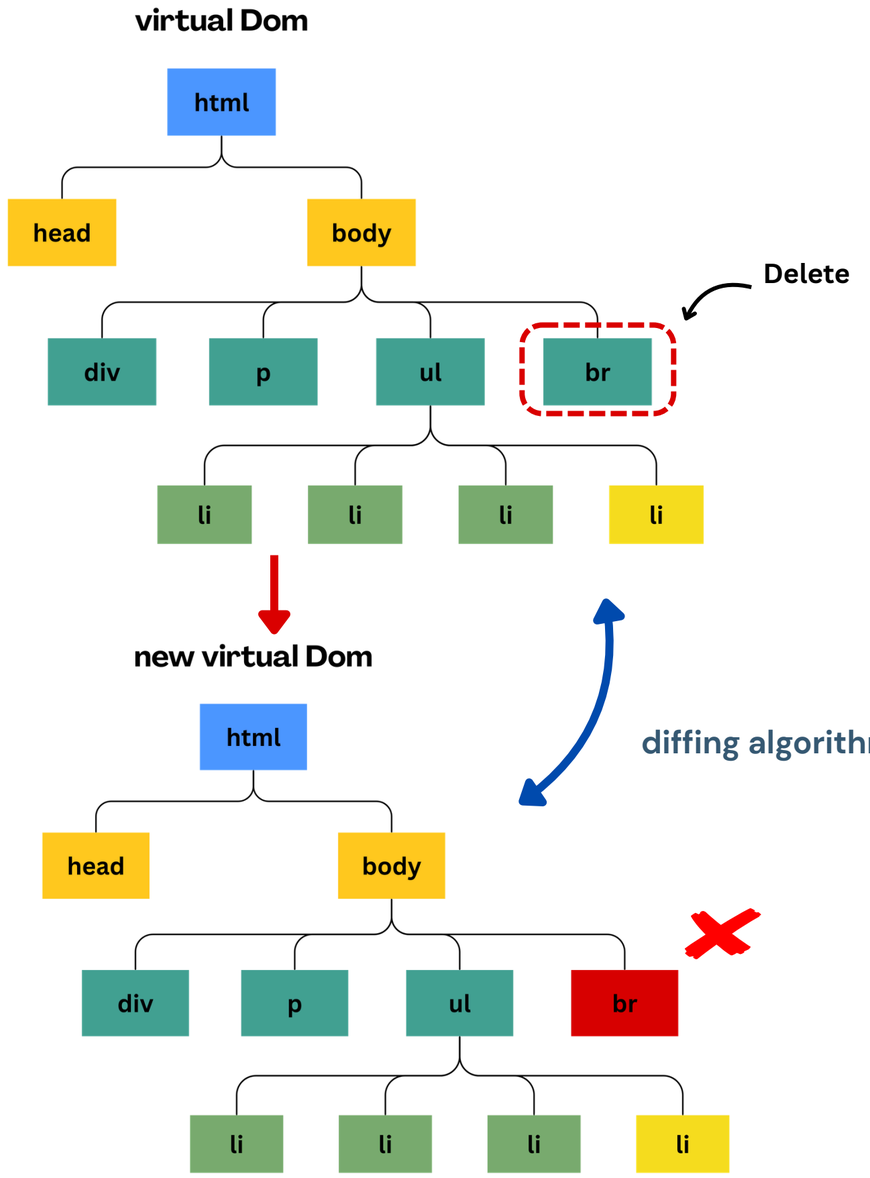
How React Works



How React Works



br deleted



diffing algorithm compares

- Reconciliation Process
- Diffing Algorithm
- Fiber Engine

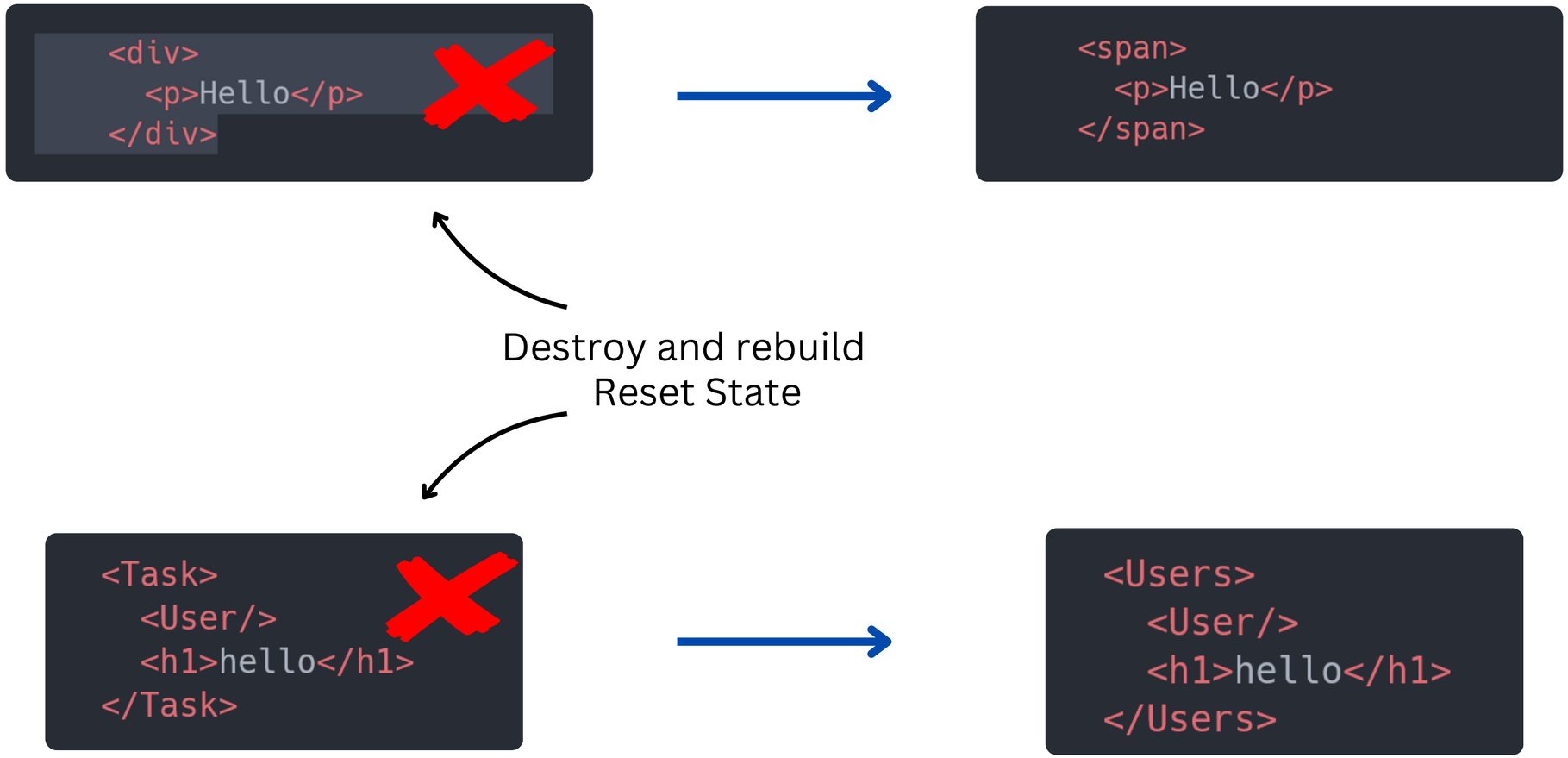
First Rule 👉 “Don’t Mutate”

Shallow comparison O(n) ✅

Deep comparison O(n)3

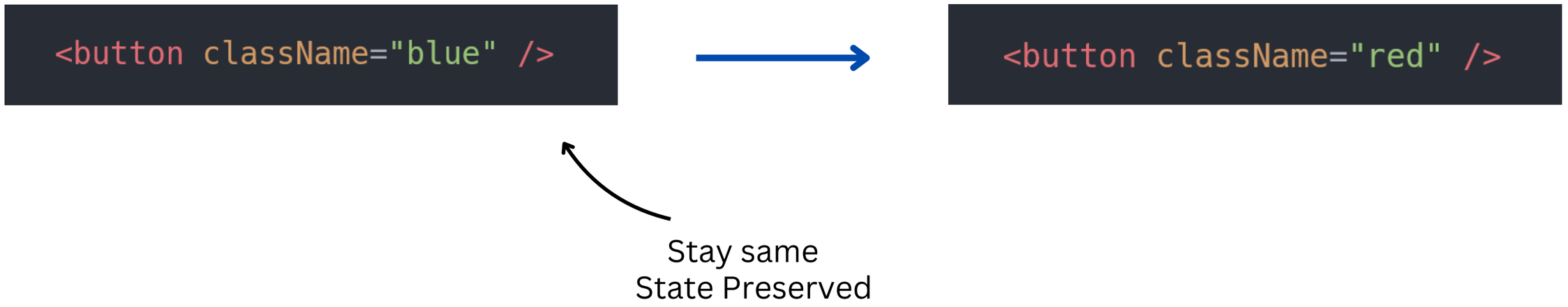
Diffing rules

1. Different Elements Same Position



Diffing rules

2. Same Elements Same Position



Component - Component instance - React Element

Component 🤔

A Component is a reusable function or class that defines how a part of the UI should look and behave.

```jsx
function User(){
  return(
    <>
      <div className="content">
        <h1>this is a user component</h1>
        <span>hello</span>
      </div>
    </>
  )
}
export default User;
```

Component Instance

A Component Instance is a specific execution of a component with its own state, props, and lifecycle.

```jsx
<User></User>
```

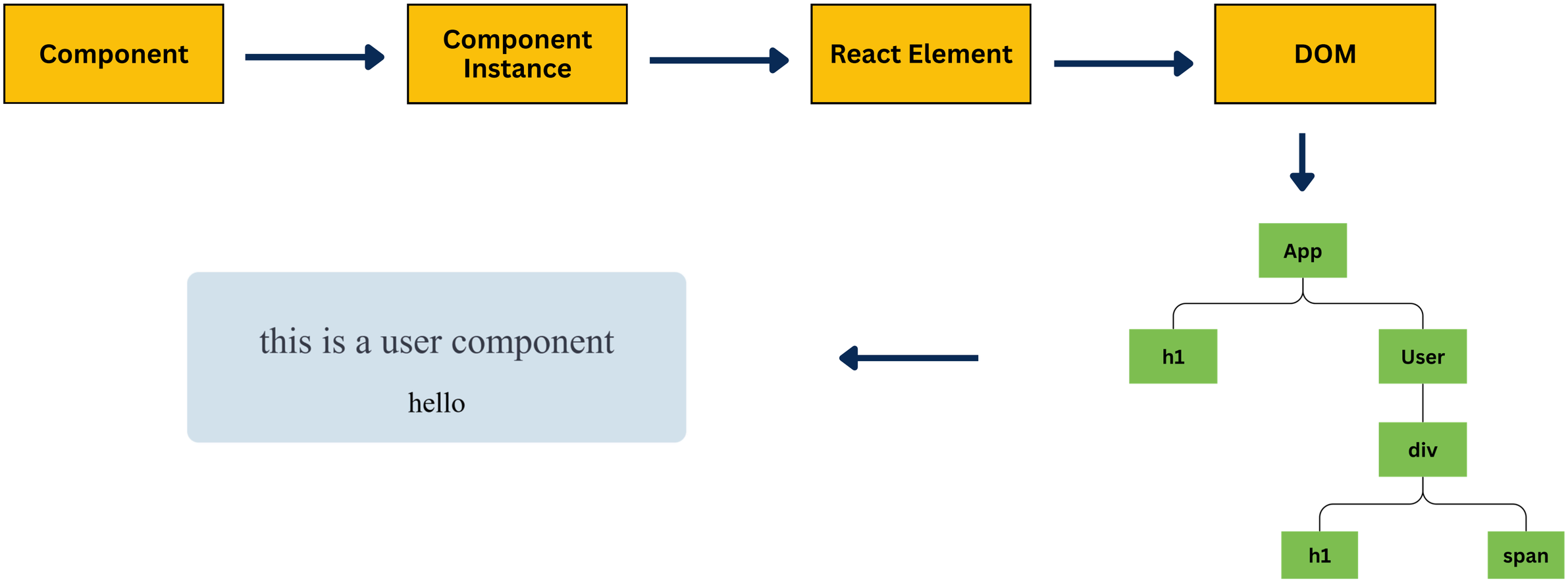
React Element (VD)

A React Element is a plain JavaScript object that describes what should appear on the U

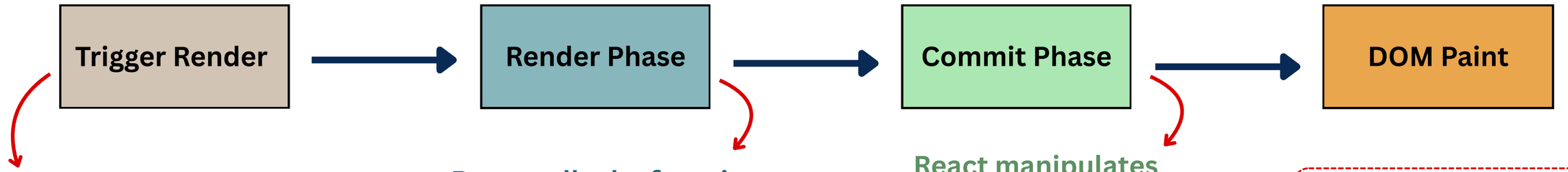
```jsx
React.createElement(
    React.Fragment,
    null,
    React.createElement(
      "div",
      { className: "content" },
      React.createElement(
        "h1",
        null,
        "this is a user component"
      ),
      React.createElement(
        "span",
        null,
        "hello"
      )
    )
  );
```

```jsx
{
  $$typeof: Symbol(react.transitional.element),
  key: null,
  props: {},
  type: function User() {},
  _owner: {tag: 0,key: null,stateNode: null,elementType: function User() {},
  _store: {validated: 0},
  ref: null,
  _debugInfo: null,
  _debugStack: "Error: react-stack-top-frame",
  _debugTask: {
    run: function () {}
  }
};
```

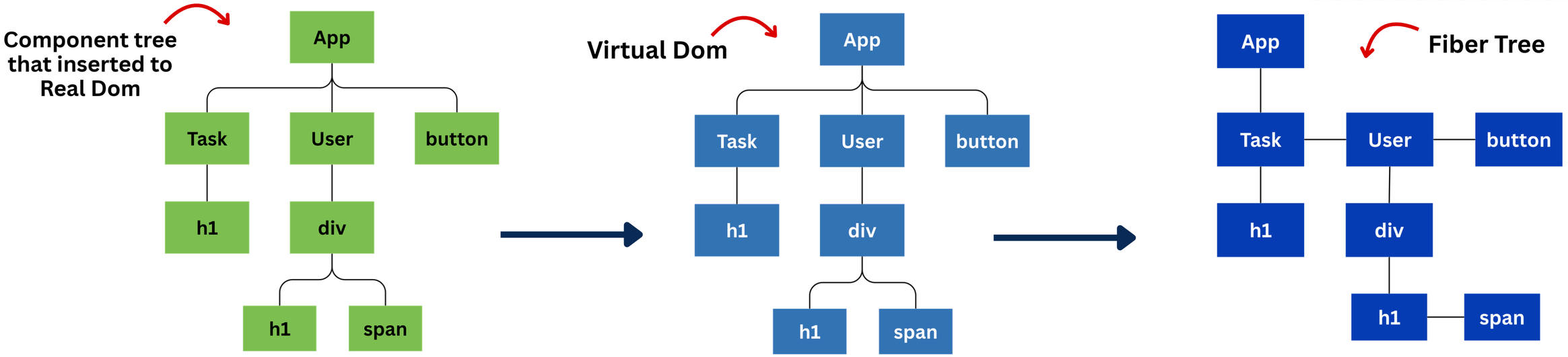
Component - Component instance - React Element



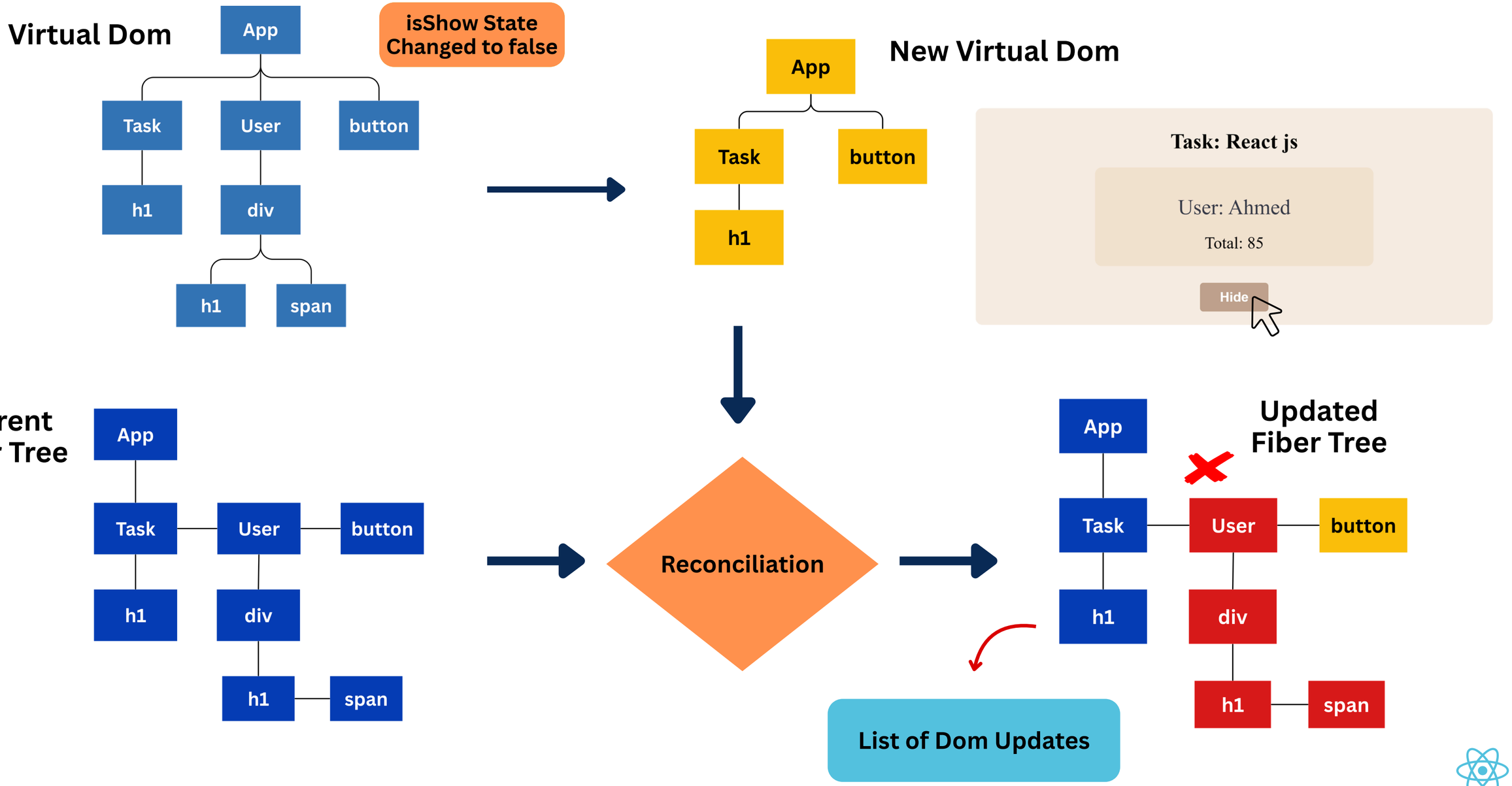
From Component To DOM



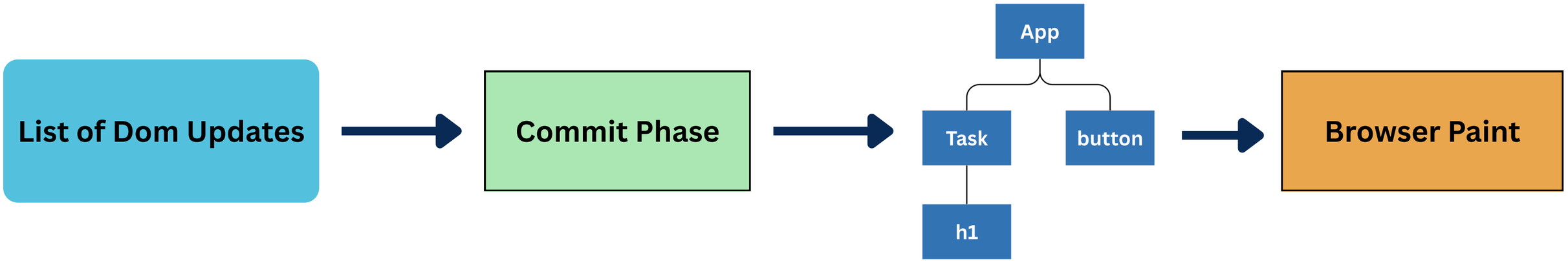
The Fiber Tree:
is React's internal data structure that represents the component hierarchy and stores all the information needed to efficiently render, update, and manage the UI.



Re-Render With Reconciliation Process



Re-Render With Reconciliation Process

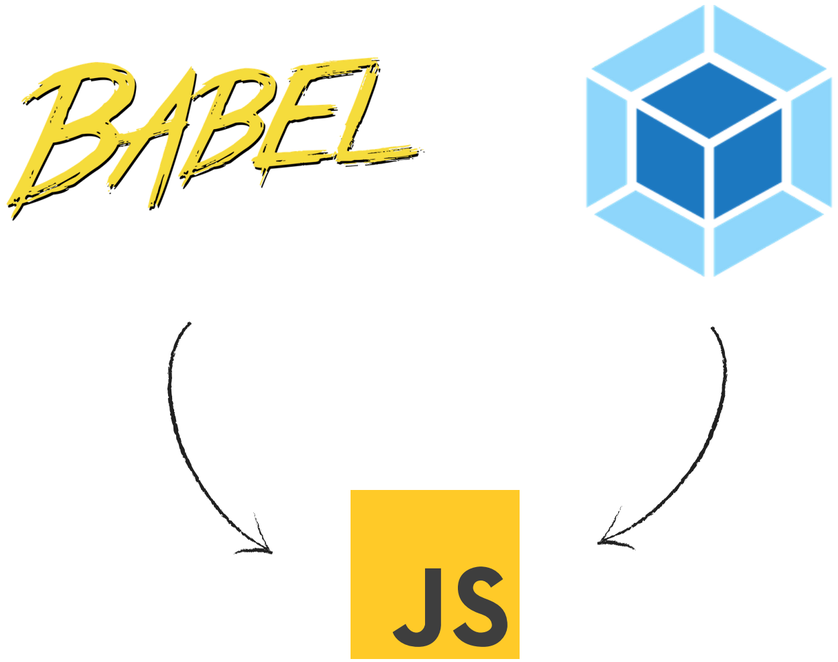


Task: React js

Show

Tools Behind

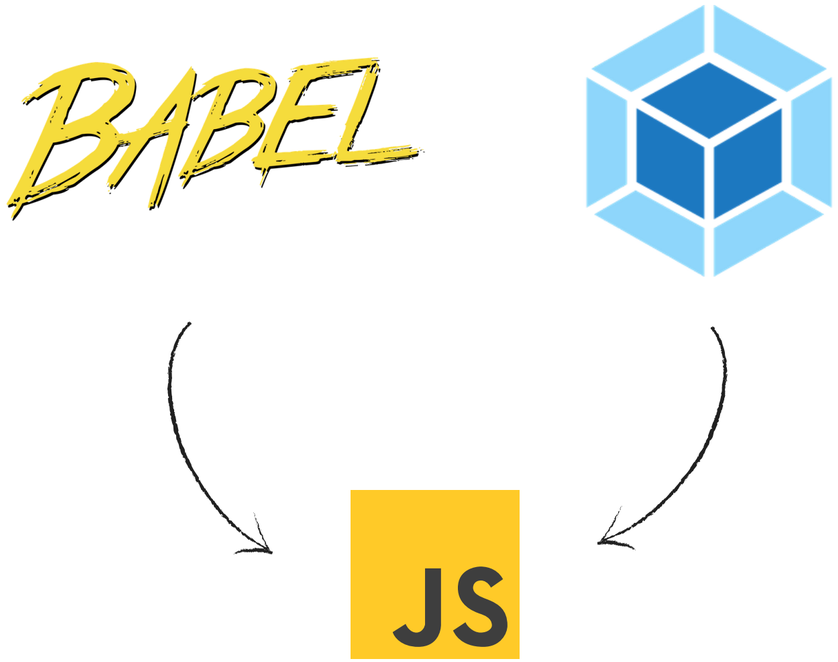
With Create-React-App Library



With Vite

Tools Behind

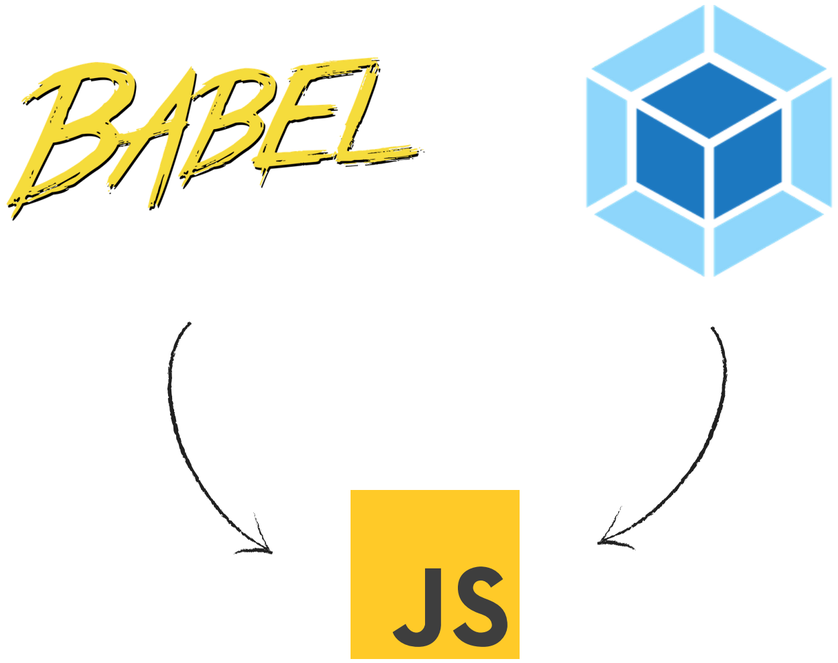
With Create-React-App Library



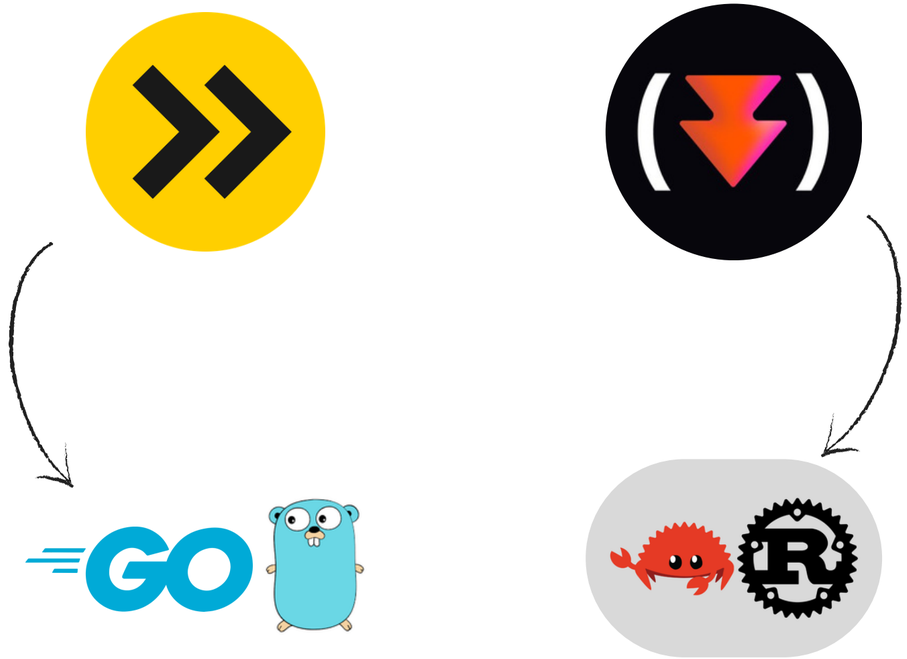
With Vite

Tools Behind

With Create-React-App Library



With Vite



Create React App

Step 1: Add a DOM Container to the HTML

First, open the HTML page you want to edit. Add an empty `<div>` tag to mark the spot where you want to display something with React. For example:

```
<!-- ... existing HTML ... -->

<div id="like_button_container"></div>

<!-- ... existing HTML ... -->
```

Step 2: Add the Script Tags

Next, add three `<script>` tags to the HTML page right before the closing `</body>` tag:

```
  <!-- ... other HTML ... -->

  <!-- Load React. -->
  <!-- Note: when deploying, replace "development.js" with "production.min.js". -->
  <script src="https://unpkg.com/react@17/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@17/umd/react-dom.development.js" crossorigin></script>

  <!-- Load our React component. -->
  <script src="like_button.js"></script>

</body>
```

```
// ... the starter code you pasted ...

const domContainer = document.querySelector('#like_button_container');
ReactDOM.render(e(LikeButton), domContainer);
```

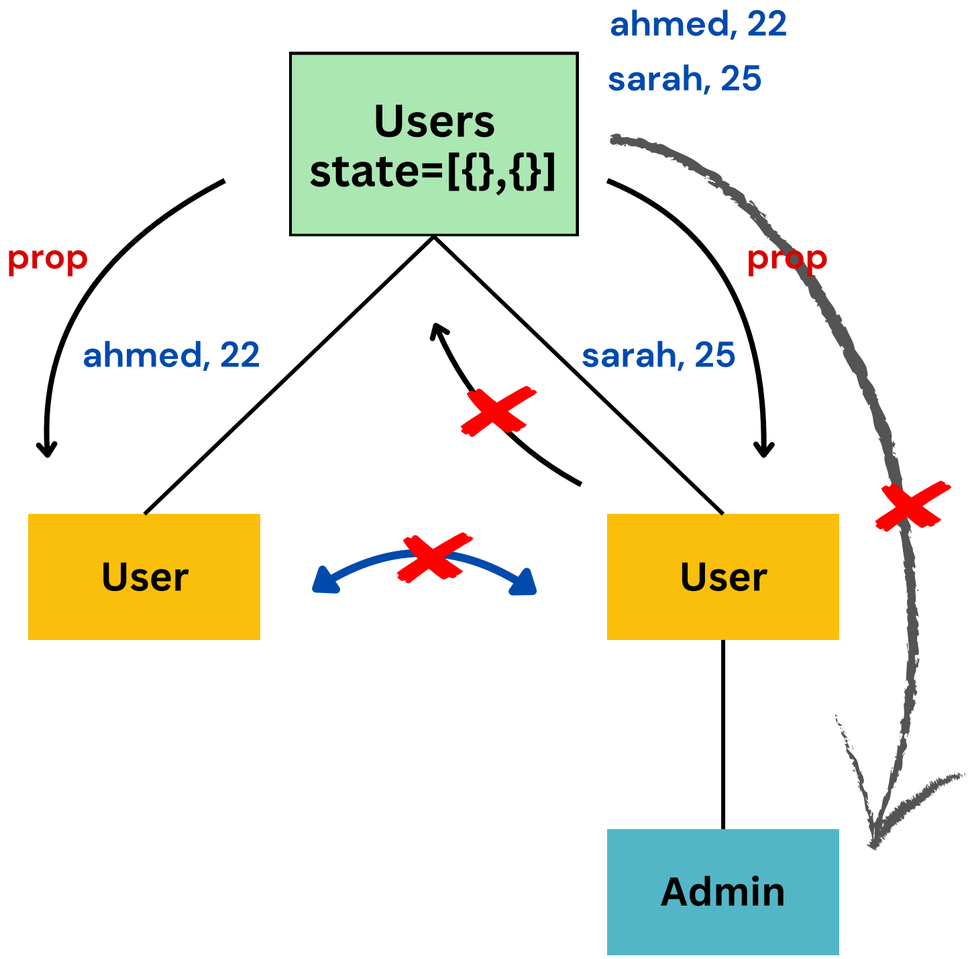
React From Scratch

Create-React-App Library

```
npx create-react-app my-app
cd my-app
npm start
```

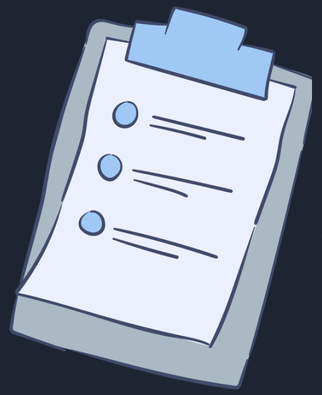
```
npm create vite@latest my-app
cd my-app
npm run dev
```

State and Props



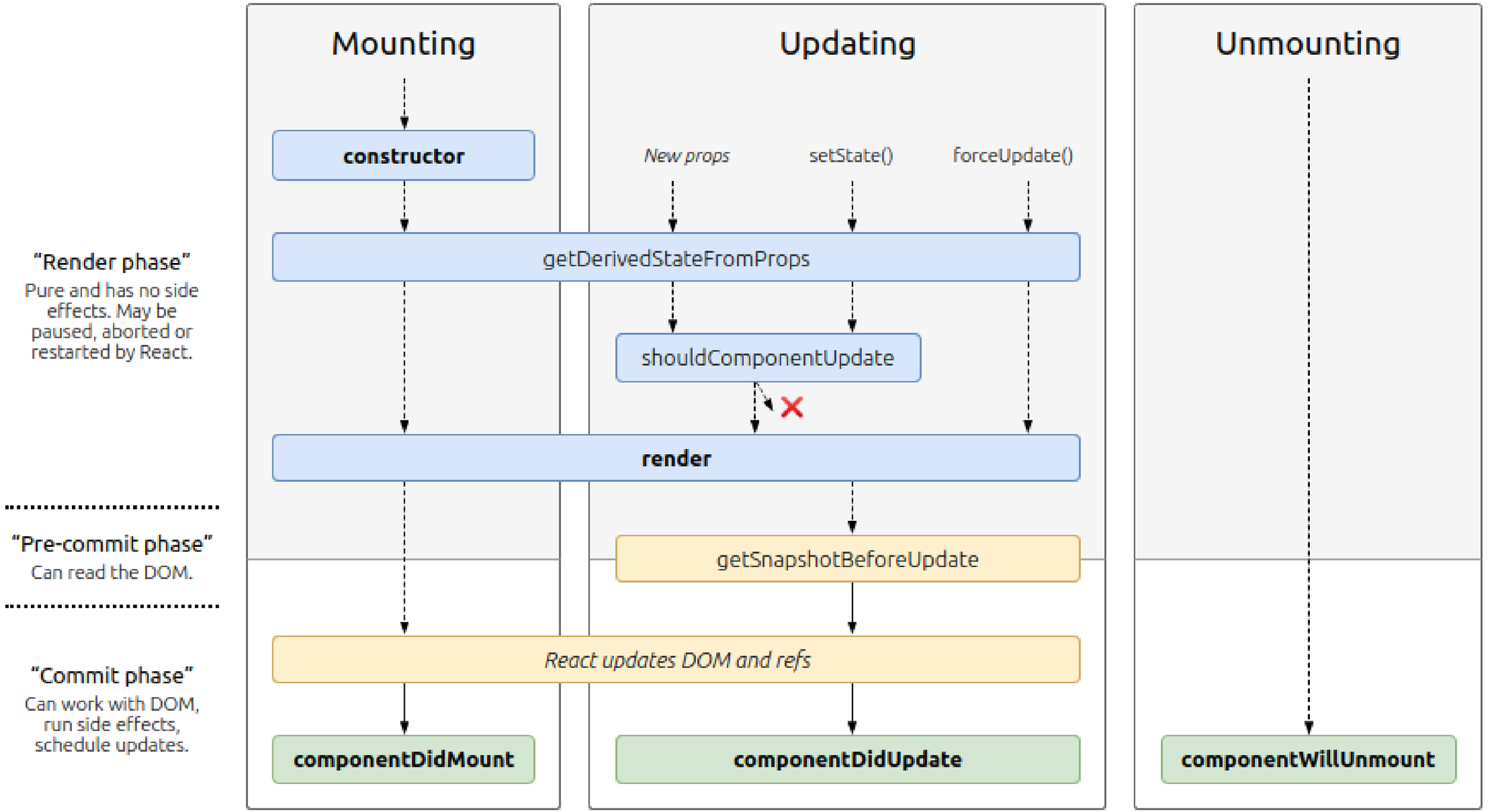
DAY 2

- Children Props
- Component Life Cycle Methods
- Events
- Call API
  - Fetch & Axios
- Dealing With Images
  - Public Folder
    - ❌ Images are not included in the build process (no optimizations)
    - ❌ You must manually ensure files exist—no Webpack/Vite validation.
  - src Folder
    - ✅ Webpack/Vite optimizes and hashes images (better Compression + Caching)
    - ✅ Ensures images exist at compile time Webpack/Vite validation

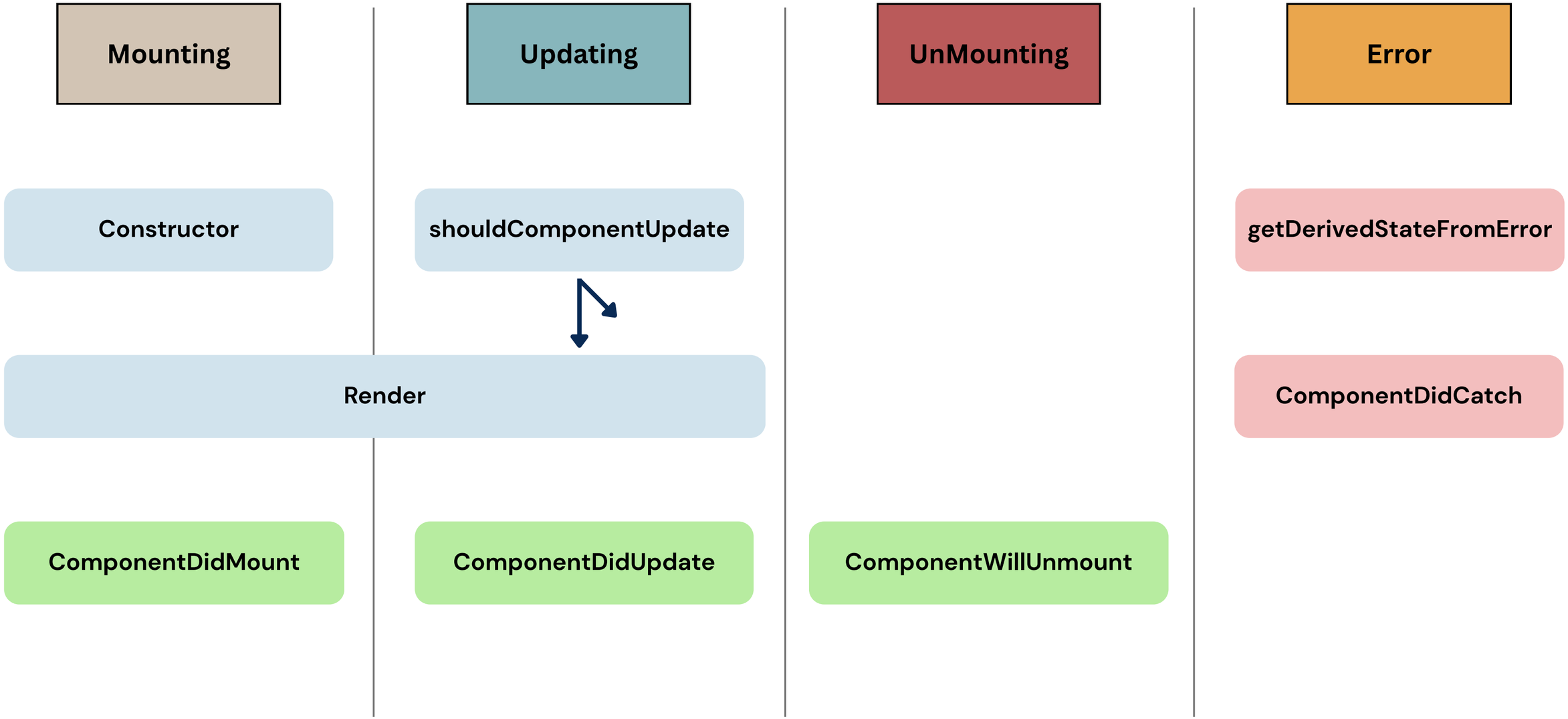


Agenda

Component Life Cycle



Component Life Cycle

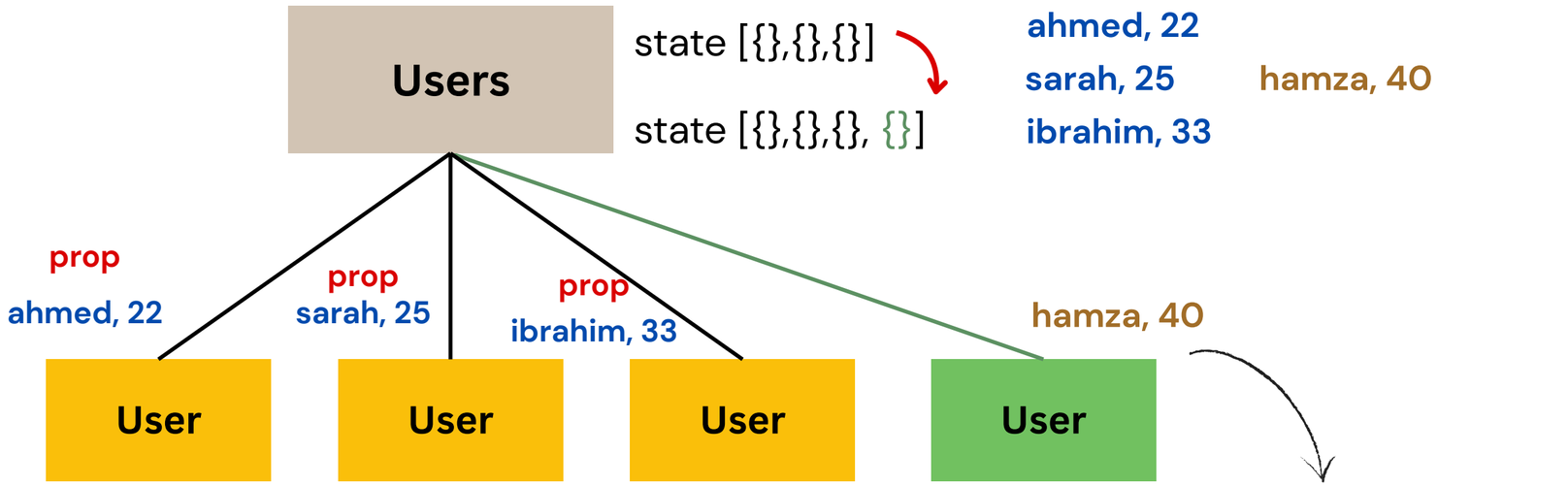


Trigger Update and Useless Rerender

Trigger Update

- state
- props
- ForceUpdate
- parent and child relationship

cause useless rerender



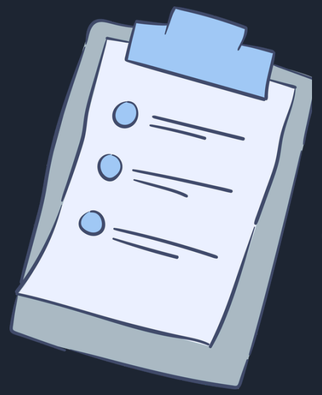
cause useless rerender

Avoid Useless Rerender

- shouldComponentUpdate(nextProps, nextState)
- PureComponent

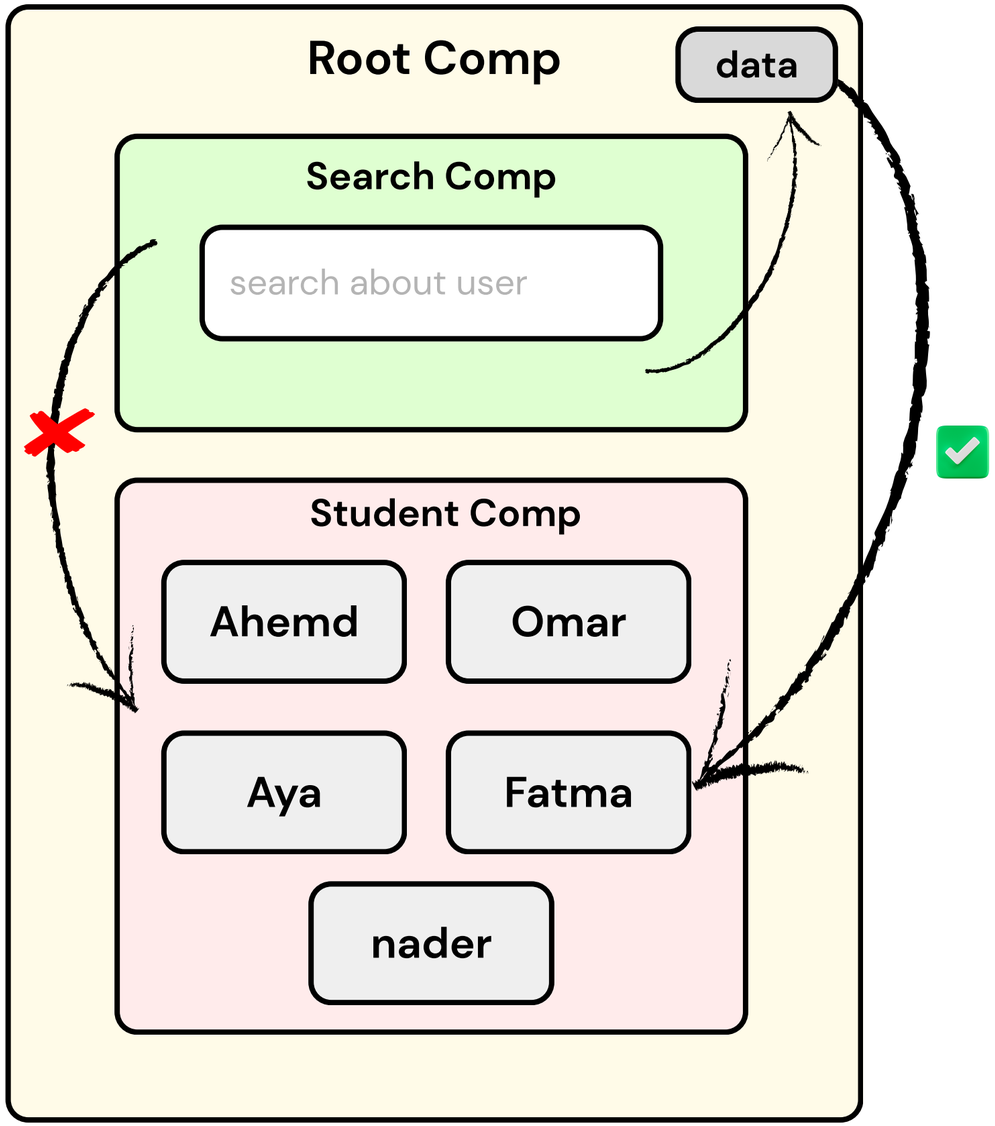
DAY 3

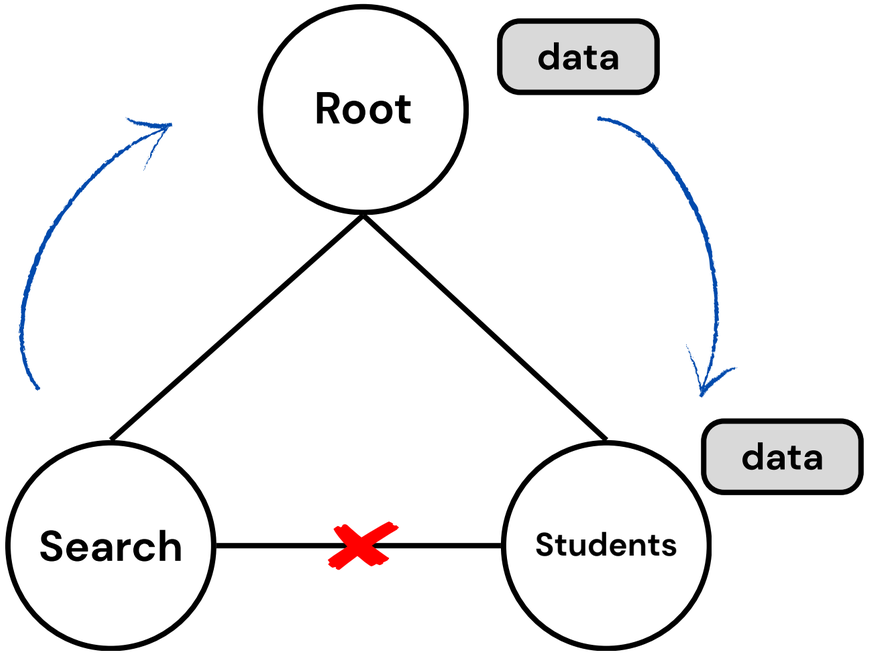
- Props Changes & Send data from child to parent
- Form
- Components Interaction
- Styling
  - Bootstrap
  - css modules



Agenda

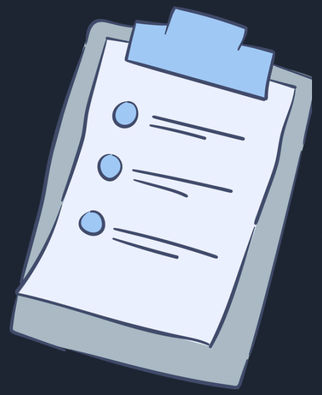
Components Interaction





DAY 4

- Intro About Hooks and history
- useState hook
  - disabled logic
- useEffect
  - intro & Take All Cases
- useCallback
- useMemo
- useRef
- useReducer
- Custom Hook



Agenda


React hooks
Introduced in React V16.8

Making Function Components its own state. Not Only that!

- Classes confuse both people and machines
- It’s hard to reuse stateful logic between components
- And more awesome powerful functionalities which we will discover some of them through our Demo
- Used in func comp or in custom hook only

THANK YOU